Certified Shortcuts and Depth Bounds for Full Optimal Binary Search Trees

Research checkpoint: exact shortcuts and barriers to relaxations

OpenAI Codex Sydney

Problem selection and sponsorship: the repository owner

October 19, 2026

Abstract

We investigate exact optimal binary search trees with ordered weights on both keys and gaps. We give sufficient key and block contraction rules, with construction cost $O(n + m^2 + \sum_t k_t^2)$ for a surviving core of m keys and contracted blocks of sizes k_t. Complementary shortcuts optimize minimum or bounded-excess unweighted path-length classes and certify unrestricted optimality with residual-weight bounds. Successful narrow-class attempts take $O(n \log n)$ work. A known split-height Monge construction supplies an $O(n^{3/2})$ attempted route at every size, with a separate certificate against taller trees.

We also implement the established short-interval/top-skeleton architecture using a variable mass contour. A directly proved mixed-weight Fibonacci bound guarantees that the contour contains every optimal subtree at a chosen depth d. If S_d intervals are admitted, exact full construction takes $O(n \log(n+1) + \min_d\{S_d + n2^d\})$ arithmetic work, including parameter selection. A cancellation argument bounds all short-table root searches by $3S_d$. On a positive family with a localized light block, the variable contour gives $O(n^{1.59024})$ work while the specified uniform width envelope requires $\Omega(n^{1.75})$ work. This is a comparison within one architecture, not a new general subquadratic bound. Dominant-key and dominant-gap certificates split the instance into independent components with local frontiers. Endpoint-certified chains take linear arithmetic work; bounded terminal components give $O(n \log n)$ work. A joint heavy-outcome certificate further isolates several comparable heavy gaps or keys, using a protected-only height bound and an $O(n \log n)$ threshold sweep.

Exact enumeration, independent cubic recurrences, adversarial inputs, and operation counts accompany all implementations. We preserve a failed mass-bound transfer and document an ordered depth relaxation, its $O(n \log n)$ oracle, and obstructions to local repairs. General worst-case construction remains quadratic; novelty and broad practical superiority are not established. The full subquadratic research objective remains open.

1 Model, scope, and prior work

Keys have fixed inorder positions $0, \ldots, n-1$ and nonnegative weights $p_0, \ldots, p_{n-1}$. The $n+1$ gaps have weights $q_0, \ldots, q_n$. A key at depth d contributes $p(d+1)$; a gap at depth d contributes qd. Charging an extra comparison at each gap adds the tree-independent constant $\sum q_i$.

For a half-open key interval $[i, j)$, write

$$W(i,j) = \sum_{r=i}^{j-1} p_r + \sum_{r=i}^{j} q_r, \qquad C(i,i) = 0.$$

The exact recurrence is

$$C(i,j) = W(i,j) + \min_{i \le r < j} \{C(i,r) + C(r+1,j)\}.$$

Knuth's monotone-root optimization computes the table in $O(n^2)$ time [1]. The central obstacle here is the number of states, not merely the cost per state. We retain both kinds of weights throughout.

Larmore gives subquadratic exact construction under restrictions on long runs of very low frequency items, together with an approximation tradeoff [2]. Karpinski, Larmore, and Rytter combine exact small intervals with a bounded top-level construction [3]. These results are essential context: an improvement on a restricted family must not be described as the first subquadratic result. This checkpoint does not make a comprehensive priority claim. The conditions below were derived in this investigation; a targeted literature search did not establish whether equivalent rules already exist.

All running-time bounds below count arithmetic operations and comparisons. They are not bit-complexity bounds for unbounded integer weights.

2 A sharper single-key contraction

A key is a *cherry* when its two children are external nodes. Replacing a cherry with one gap is exact: if the key and its gaps have weights p, x, y, their total is $s = p + x + y$. A cherry rooted at depth d contributes $s(d+1)$, whereas the replacement gap contributes sd.

Theorem 1 (Single-key rule)**.** *Let a key of weight p have adjacent gap weights x, y. If its immediate predecessor and successor keys have weights a, b and*

$$\min(a,b) \;\ge\; 2p + x + y,$$

there is an optimal tree in which this key is a cherry. At the left endpoint, it suffices that the successor weight is at least $p + x$; at the right endpoint, it suffices that the predecessor weight is at least $p + y$. Consequently contraction to a gap of weight $s = p + x + y$ satisfies $\mathrm{OPT}(\textit{original}) = \mathrm{OPT}(\textit{contracted}) + s$.

Proof. Consider the key at depth d in any tree. If its left subtree is nonempty, the immediate predecessor has depth $d + l$, $l \ge 1$. If its right subtree is nonempty, the immediate successor has depth $d + r$, $r \ge 1$. Set the corresponding distance to zero for an empty internal side.

When $l \ge 1$, move the predecessor to the chosen key's position, replacing the predecessor's old position by its left child. Insert the chosen key as a cherry at the leftmost gap of the former right subtree. Inorder order is preserved. All other outcomes either retain their depths or become shallower. The predecessor saves la, and the cost change is at most

$$-la + (r+1)p + (r-l+1)x + y. \tag{1}$$

The mirrored successor exchange has change at most

$$-rb + (l+1)p + x + (l-r+1)y. \tag{2}$$

If both sides are nonempty and $l \ge r$, substituting $a \ge 2p + x + y$ in (1) gives

$$(r+1-2l)(p+x) + (1-l)y \le 0.$$

Use (2) when $r \geq l$. If only the left side is nonempty, $r = 0$ and $a \geq p + y$ makes (1) nonpositive. The other case is symmetric. The stated interior condition implies these one-sided conditions. At a global endpoint only the appropriate one-sided case is possible. With both sides empty the key is already a cherry.

Applying the exchange to an optimum proves the existence claim. Contraction subtracts exactly s. Conversely, every contracted tree expands to a valid original tree at additional cost s, proving equality. □

The weaker condition $\min(a, b) \geq p + x + y$ is false. For $p = (24, 14, 24)$ and $q = (1, 0, 9, 0)$, the middle key has local mass 23 and neighbors of weight 24. Contracting it forces cost 142, while the unrestricted optimum costs 130.

Maintain a doubly linked list of remaining keys and weighted gaps. Initially queue every key; after a contraction, only its predecessor and successor need rechecking. At most $3n$ queue entries are processed. Saving the contracted tree at the new gap permits reconstruction. Thus this pass takes $O(n)$ operations and space, and a Knuth solve on the remaining m keys gives total time $O(n + m^2)$. No confluence or minimum-core-size property is claimed.

3 Whole-block contraction

Let $B = [i, j)$ be a nonempty proper interval, including its endpoint gaps q_i, q_j. Put $W = W(i, j)$ and $C = C(i, j)$, with costs measured from a stand-alone root of depth zero.

Theorem 2 (Block rule)**.** *If both boundary keys exist and their weights satisfy*

$$p_{i-1} \geq W + C, \qquad p_j \geq W + C,$$

there is an optimum in which all of B forms one subtree. For a prefix or suffix block, it suffices that the sole boundary key has weight at least C. Contracting the block to one gap of weight W then gives

$$\mathrm{OPT}(\textit{original}) = \mathrm{OPT}(\textit{contracted}) + C.$$

Proof. In any tree, let v be the lowest common ancestor of gaps q_i, q_j, at depth d. Its key lies in B, and every outcome of B descends from v. Hence the original contribution of B is at least $(d+1)W$.

Within the subtree rooted at v, delete the keys of B from the inward fringes of its two children. On the left, a deleted key has no retained right child and can be suppressed in favor of its left child; on the right the process is symmetric. Let the retained outside trees be A, D. All their retained outcomes are at depths no greater than before. If A is nonempty, its maximum is boundary key $i - 1$; if D is nonempty, its minimum is boundary key j.

Suppose both are nonempty, and their boundary keys had original depths $d + l, d + r$. If $l \geq r$, remove the maximum key from A, replacing its position by its left child, and make that key the new root at v. Its left child is the remainder of A and its right child is D. Put an optimal tree for B at the leftmost external position of D. This position has depth at most $d + r + 1$. No outside outcome other than the promoted boundary moves deeper, and that boundary saves lp_{i-1}. The increase in the contribution of B is at most

$$C + (d + r + 1)W - (d + 1)W = C + rW.$$

Since $l \geq r \geq 1$ and $p_{i-1} \geq W + C$, the saving is at least $l(W + C) \geq rW + C$. The mirrored construction applies for $r \geq l$.

If only one retained outside tree is nonempty, promote its inward boundary key to v and attach the optimal block as the other child. The block increase is at most C, while that boundary saves at least its own weight. The interior hypothesis suffices, and for a global prefix or suffix the weaker stated hypothesis suffices. If both outside trees are empty, B already occupies the subtree at v and can be optimized in place.

The construction never increases cost. Apply it to an optimum. A block at depth e contributes $C + eW$, while its replacement gap contributes eW. Contraction and expansion now prove the equality. □

A balanced k-key tree has height $h_k = \lceil \log_2(k+1) \rceil$ and cost at most $h_k W$. Thus the interior test

$$\min(p_{i-1}, p_j) \geq (1 + h_k)W$$

and endpoint test $p_{\text{boundary}} \geq h_k W$ certify a block without first solving it. These are sufficient conditions, not characterizations.

4 An adaptive linear selection pass

Build the maximum Cartesian tree of the key-weight sequence, breaking ties deterministically. Its subtree intervals form a laminar family: intervals are disjoint or nested. Compute their sizes and masses by one traversal and prefix sums. Test each proper interval with the balanced-tree bound above. This inspects only $O(n)$ candidates. The restriction to Cartesian intervals is an algorithmic choice, not a claim that all useful blocks occur there.

For a threshold b, select the maximal certified intervals whose sizes are at most b. Different selected intervals have a surviving separator key between them. Contracting one therefore leaves every other selected interval's weights and boundary-key weights unchanged, so the block theorem applies successively. Write their sizes as $k_1, \ldots, k_s$, and $m = n - \sum_t k_t$. The exact number of nonempty interval-DP states is

$$S(b) = \tau(m) + \sum_t \tau(k_t), \qquad \tau(u) = \frac{u(u+1)}{2}.$$

We choose a threshold minimizing $S(b)$ without running these DPs. For every certified interval I of size k, find its maximal proper certified descendants J. Immediately before I becomes eligible, these descendants are its selected blocks. Its activation therefore changes the core size and block-state total by

$$\Delta m = -\left(k - \sum_J |J|\right), \qquad \Delta S_{\text{blocks}} = \tau(k) - \sum_J \tau(|J|).$$

These sums are computed bottom-up. Bucket activation events by integer size $1, \ldots, n$ and scan the buckets, maintaining m and the block-state total. There are $O(n)$ events and buckets. Equal-size intervals are disjoint; evaluate a threshold after processing its entire bucket. The empty selection $b = 0$ is included.

Corollary 3. *The selected threshold and blocks are found in $O(n)$ arithmetic operations and space. Solving the blocks and surviving core by Knuth's method and expanding their saved trees takes*

$$O\left(n + m^2 + \sum_t k_t^2\right)$$

time and $O(n + m^2 + \max_t k_t^2)$ space when block tables are released after each solve. The chosen state count is no larger than $n(n+1)/2$. Worst-case time remains $O(n^2)$.

The minimization is only over these size thresholds in this candidate hierarchy, not over every possible partition. It does not minimize wall-clock time. Preprocessing, table allocation, root scans, and reconstruction still count as work.

For perspective, a certified partition with $m = O(n^{2/3})$ surviving keys and every $k_t = O(n^{1/3})$ gives $O(n^{4/3})$ time, since $\sum k_t^2 \le (\max k_t)n$. This is a conditional structural consequence, not an unrestricted bound.

5 Recursive construction with a bounded preprocessing depth

The implementation `recursive_contract.py` combines the proved components. At each call it applies the single-key pass, compares the order-free lower bound of Section 13 with a feasible bisection-tree cost, and returns immediately on equality. Otherwise it selects blocks as above. It recurses only if their combined table-state estimate is at most three quarters of the full estimate; small components and unsuccessful decompositions fall back to Knuth's method.

Suppose a component has $m > 8$ keys after the single-key pass and is successfully decomposed. Every child component of size s satisfies $s(s+1) \le \frac{3}{4}m(m+1)$, hence $s < 0.92m$. Child sizes sum to m. Consequently the recursion has logarithmic depth, and the total number of keys processed on any recursion level is at most n. Linear selection and reconstruction cost $O(n \log n)$ in total. The heap-based equality checks cost $O(n \log^2 n)$ over all levels. If the terminal DP sizes are r_t, the construction therefore takes

$$O\left(n \log^2 n + \sum_t r_t^2\right)$$

arithmetic operations. Its worst case is still quadratic. This threshold prevents repeated expensive scans after arbitrarily small reductions. Correctness follows by contraction, exact equality certificates, and induction on the strictly smaller recursive components; reconstruction restores the original key indices and gap subtrees.

The recursive implementation passed 3,000 further random inputs through $n = 30$ against the cubic oracle and direct tree evaluation. A separate nested fixture checks composition across multiple decomposition levels. Matched Python timings use five inputs per family, alternate which solver runs first, and include preprocessing and reconstruction for both solvers. At $n = 1024$, the median runtime ratio (recursive divided by Knuth) was 0.134 on the positive-weight heavy-separator family; its median terminal DP count was 30,017 versus 524,800. On random mixed weights the ratio was 0.986 and on near-uniform weights 1.001, which support no meaningful gain. At $n = 512$, the respective random and near-uniform ratios were 1.131 and 1.093: preprocessing can make the method slower. These are exploratory measurements on one M4 host, not production-language or cross-machine claims.

6 A certified shortcut through nearly complete shapes

Let $I(T)$ be the sum of internal-node depths, with root depth zero. The external path length is $I(T) + 2n$, by induction on the two subtrees. Consequently uniform key weight a and gap weight b give cost

$$(a + b)I(T) + (a + 2b)n.$$

For an arbitrary instance set $a = \min_i p_i$, $b = \min_j q_j$, and let H' be the order-free mixed-node Huffman bound of Section 13 on residual weights $p_i - a, q_j - b$. These weights remain nonnegative. Write $I_*(n)$ for the minimum possible unweighted internal path length, and let C_* be the best weighted cost among all trees with $I(T) = I_*(n)$.

Theorem 4 (Minimum-path-length certificate)**.** *For $n \geq 1$, if*

$$C_* \;\leq\; (a + b)(I_*(n) + 1) + (a + 2b)n + H',$$

then a tree attaining C_* is a globally optimal full OBST.

Proof. Split each outcome weight into its uniform baseline and residual. Every tree outside the minimum-I class has integer $I(T) \geq I_*(n) + 1$, and its residual cost is at least H'. Its full cost is therefore at least the right-hand side. Within the class, C_* is minimal by definition. Equality is sufficient, including when $a + b = 0$. □

Optimize the class without interval tables. Put $h = \lfloor \log_2(n + 1) \rfloor$, $S = 2^h$, $m = S - 1$, and $r = n - m$. Every minimum-I tree has the complete upper skeleton with m keys, and expands exactly r of its S external slots into single-key cherries. Indeed, a missing shallower slot could otherwise receive a deepest internal leaf, strictly reducing I. Inorder relabeling preserves search-tree feasibility in this unweighted shape argument. Thus

$$I_*(n) = (h - 2)2^h + 2 + hr.$$

Process skeleton slots $j = 0, \ldots, S - 1$ in inorder. State (j, e) records the number of processed slots and expansions, so the next input index is $i = j + e$. An unexpanded slot charges hq_i; an expanded slot charges $(h+1)(p_i+q_i+q_{i+1})$. Except after the last slot, also charge the next skeleton key at comparison length

$$d_j = h - v_2(j + 1),$$

using p_i after an unexpanded slot and p_{i+1} after an expansion. Here v_2 counts trailing zero bits. Exactly r expansions are required, so state (j, e) is retained only when

$$\max(0, r - (S - j)) \leq e \leq \min(r, j).$$

With $s = \min(r, S - r)$, each stage has at most $s + 1$ states and each state has two outgoing choices. This computes C_* and backpointers in $O(n(1 + s))$ time and space. A linear Cartesian-tree pass converts the chosen comparison lengths into the labeled tree. No key or gap weights are discarded in these transitions.

There is an exact early rejection test. Let F be the cost obtained by assigning independently sorted key and gap weights to the shallowest depths of a minimum-I shape. This is feasible for the order-free residual relaxation, so $H' \leq F - [(a + b)I_* + (a + 2b)n]$. Therefore $C_* > F + a + b$ proves the certificate cannot succeed. Otherwise the sorted lists are reused in a two-queue Huffman implementation: nonnegative weights and nondecreasing internal weights make the sequence of

merged weights nondecreasing. This adds linear work after sorting. The queue optimization is standard and is not claimed as novel.

The implementation attempts this DP only when $s \le \lfloor \log_2 n \rfloor + 1$. Including the residual bound, preprocessing, and reconstruction, a successful attempt costs $O(n \log n)$ arithmetic operations and space. A skipped or failed attempt falls back to Knuth, or to the preceding recursive construction. Both versions remain exact and have quadratic worst-case time. The test is a certificate, not an assumption that near-uniform inputs have balanced optima. Novelty of this shortcut has not been established.

Verification and limits. Minimum-I optimization matched independent enumeration of all tree shapes through eight keys on 360 weighted inputs. Both fallback versions matched independent cubic DP and direct tree costs on 3,000 further mixed-weight inputs through thirty keys; the unrestricted certificate succeeded on 768. At $n = 1,024$, one seeded input with every weight in $\{999, 1000, 1001\}$ used 2,048 shape states, 3,070 transitions, 10,511 sorting comparisons, and 1,350 merge comparisons, certifying cost 19,479,590. The nonminimum-I lower bound was 19,480,123. Knuth used 524,800 states and 1,050,139 root candidates. These work units have different costs; the state ratio is not a timing claim.

The selected perturbation sweep certified amplitude 8 around 1,000 at $n = 256$, amplitude 4 at $n = 512$, and amplitude 1 at $n = 1,024$; larger tested amplitudes at those sizes failed. These are seeded observations, not universal thresholds. Ordinary random inputs failed the certificate. Sizes 384, 768, and 1,000 deliberately skip the shortcut because their minimum-I shape space exceeds the logarithmic width budget.

Six matched trials per fixture used all six orderings of Knuth, recursive contraction, and the combined solver, with identical inputs and correctness checks outside timing. On Apple M4 with Python 3.9.6, median per-trial combined/Knuth time ratios at $n = 1,024$ were 0.03934 for near-uniform, 0.10478 for heavy separators, 1.05454 for random, 0.91929 for log-uniform, and 1.03117 for gap spikes. All six near-uniform cases certified. The skipped near-uniform controls at $n = 768$ and $n = 1,000$ gave ratios 1.17886 and 1.04896. At $n = 512$ the combined method was about 21% slower than recursive contraction alone on heavy separators. These exploratory timings include sorting, lower bounds, and reconstruction; they do not establish broad superiority. The final early-rejection and two-queue implementations were also checked against the original heap certificate on all 3,000 inputs.

Optimize the baseline rather than fix it. The largest subtractable baseline need not give the strongest certificate. Set $a_0 = \min p_i$, $b_0 = \min q_j$, and use $a = a_0 - \delta$, $b = b_0 - \delta$ for an integer $0 \le \delta \le \lfloor (a_0 + b_0)/2 \rfloor$. The residual weights remain nonnegative and $a + b \ge 0$. Although one baseline term may become negative, it is only an algebraic decomposition; the input probabilities are unchanged. Let H_δ denote the residual order-free optimum. The exclusion bound is

$$B(\delta) = (a_0 + b_0 - 2\delta)(I_* + 1) + (a_0 + 2b_0 - 3\delta)n + H_\delta.$$

Every tree outside the minimum-I class costs at least $B(\delta)$. Thus any sampled price with $C_* \le B(\delta)$ certifies the same candidate.

For a fixed residual tree, increasing δ adds one unit to every key and gap weight, changing its cost at rate $2I + 3n$. Consequently B is a pointwise minimum of affine functions, hence concave. A minimizing residual tree supplies a supergradient

$$g = 2(I - I_* - 1), \qquad B(x) \le B(\delta) + g(x - \delta).$$

If $g > 0$, smaller prices cannot improve the current value; if $g < 0$, larger prices cannot improve it. If $g = 0$, the value is globally maximal. Binary interval elimination, retaining the best sample, therefore finds the maximum over allowed integer prices when run to completion.

The same linear oracle supplies I. Give every original outcome secondary weight1, order queue items lexicographically by residual mass and number of original outcomes, and sum both merge coordinates. The first gives H_δ and the second gives $2I+3n$. This is the Huffman exchange argument with an infinitesimal positive common perturbation, choosing minimum I among residual-cost ties. The weight order is independent of δ, so no additional sorting is needed. Each price costs $O(n)$.

The fixed-profile ceiling extends to all prices: $B(\delta) \le F + a_0 + b_0 - 2\delta$. Therefore an input rejected by the existing profile test cannot be rescued by this search. The implementation tries prices only after the old certificate fails while passing that test, and permits at most $\lfloor \log_2 n \rfloor + 1$ extra oracle calls. This preserves the $O(n \log n)$ attempted-shortcut bound. Reaching the cap does not establish that the best price was found; every sampled certificate is still valid, and failure invokes the exact fallback.

The lexicographic oracle matched 400 exhaustive shape-profile comparisons through eight keys. Uncapped search matched exhaustive evaluation of all allowed integer prices on 1,200 inputs. On 3,000 additional full-OBST inputs through thirty keys, both capped fallback versions matched cubic DP and direct tree costs; 87 previously unsuccessful unrestricted certificates were recovered. A separate seeded sweep recovered the $n = 1,024$, amplitude 2 input with two additional oracle calls, and the $n = 2,048$, amplitude 1 input with three. Larger tested perturbations usually exceeded the profile ceiling. This applies standard concave-price optimization to the certificate; it is not a general subquadratic result.

In the seeded 2,048-key case, the candidate costs 43,032,446; the bound rises from 43,031,347 at $\delta = 0$ to 43,032,459 at $\delta = 125$. The three extra passes use 6,144 contractions and 6,141 pair comparisons, in addition to 4,096 shape states, 6,142 transitions, 20,784 sorting comparisons, and 2,752 original merge comparisons. Knuth uses 2,098,176 interval states and 4,198,710 root candidates on that input.

A separate six-trial comparison uses all orderings of Knuth, the old hybrid, and the price-search hybrid. All six $n = 1,024$, amplitude 2 cases and all six $n = 2,048$, amplitude 1 cases were newly certified. Median per-trial price-search/Knuth ratios were respectively 0.04953 and 0.06128. At $n = 2,048$, amplitude 2 still failed, with ratio 0.99391. Ordinary random and gap-spike controls at $n = 1,024$ gave ratios 1.06492 and 1.12941. The already-certified $n = 1,024$, amplitude 1 control made no extra oracle calls yet showed a price-search/old-hybrid ratio of 1.13564. This variability limits interpretation of small timing differences; operation counts and all raw observations are retained. The host was Apple M4 with Python 3.9.6.

7 Exact bounded-excess construction by dyadic intervals

The candidate class can be expanded beyond minimum internal path length. This is necessary even on some mildly nonuniform inputs. For $n = 255$, set all gap weights to 16 and all key weights to 16, except keys $8j + 1$ which have weight 17. The unique minimum-I tree costs 61,680; Knuth gives optimum 61,656, attained with $I = I_* + 2$. The algorithm below constructs and certifies this optimum. It retains both kinds of weight and never assumes that the restricted optimum is globally optimal.

An exact representation. An ordered full binary tree is a tiling of $[0, 1]$ by consecutive aligned dyadic intervals: a gap at depth d has width 2^{-d}. A leaf starting at x must satisfy $x/2^{-d} \in \mathbb{Z}$.

Conversely such a tiling is a complete prefix code and determines an ordered full tree. Alignment, not just the Kraft equality, enforces order. On a grid of height H, an interior boundary at integer coordinate Y corresponds to a key with comparison length $H - \nu_2(Y)$. Thus gap and key costs are both local in a left-to-right tiling construction.

Theorem 5 (Bounded-excess construction)**.** *For $n \geq 1$, let $N = n + 1$, $h = \lfloor \log_2 N \rfloor$, $S = 2^h$, $r = N - S$, and $s = \min(r, S - r)$. For any nonnegative integer budget k, the minimum full weighted cost among all trees with $I \leq I_* + k$ can be found and reconstructed in*

$$O\big(n(s + k + 1)(k + 1)^2 2^k\big)$$

arithmetic operations, using $O\big(n(s + k + 1)(k + 1)2^k\big)$ space for backpointers.

Proof. Measure dyadic widths in units of 2^{-h} and put $w = 2^{h-d}$. For integral depth d, the potential

$$f(d) = d - h - 2 + 2^{h+1-d} = d - h - 2 + 2w$$

is nonnegative, with zeros exactly at h and $h + 1$. The minimum external path length is $hN + 2r$. Since external path length equals $I + 2n$ and $\sum_i w_i = S$, every full tree satisfies

$$\sum_i f(d_i) = \sum_i d_i - (h + 2)N + 2S = I - I_*.$$

Every depth in the budgeted class therefore has $f(d) \leq k$. This implies $d \leq H = h + k + 2$, and only $O(k + 1)$ integral depths are possible. For integral d, direct substitution gives

$$\tfrac{1}{2} - \tfrac{1}{2} f(d) \leq w \leq 1 + f(d).$$

Apply these inequalities to a prefix of i leaves and to its remaining suffix, each having potential at most k. The cumulative position y must obey

$$\max\left(\tfrac{i-k}{2}, i - r - k\right) \leq y \leq \min\left(i + k, \tfrac{S-r+i+k}{2}\right).$$

This interval has width at most both $r + 2k$ and $(S - r)/2 + k$, hence $O(s + k)$.

Set $K = 2^{k+2}$. A state after i leaves stores integral drift $u = K(y - i)$ and spent potential $z = K \sum f(d)$. For depth d, let $W = 2^{H-d}$. Require $(iK + u) \bmod W = 0$ and update

$$u' = u + W - K, \qquad z' = z + K(d - h) + 2(W - K).$$

Retain the position band and $0 \leq z' \leq Kk$. Charge $q_i d$ and, for $i > 0$, charge $p_{i-1}(H - \nu_2(iK + u))$. Start at $(u, z) = (0, 0)$ and finish after N leaves with $u = -rK$. Every retained complete path is an aligned full tiling with excess at most k; every tree in the class passes the same tests. The DP therefore optimizes exactly this class. For fixed u, the congruence $z \equiv 2u \pmod{K}$ leaves at most $k + 1$ possible spent potentials. There are $O((s + k + 1)2^k)$ positions per stage and $O(k + 1)$ choices per state, proving the bounds. The bands admit direct array indexing; the Python prototype uses sparse dictionaries. Backpointers recover gap depths; the boundary lengths reconstruct the tree in linear time. □

Perfect-size refinement. When $N = S$, use instead $f_0(d) = d - h + \frac{3}{2}(2^{h-d} - 1)$. It is nonnegative on integer depths, vanishes only at h, and also sums to $I - I_*$. The inequality $|2^{h-d} - 1| \leq 2 f_0(d)$ and zero total drift imply $|y - i| \leq k$. Here $H = h + k + 1$, $K = 2^{k+1}$, and $z = 2K \sum f_0$. Its increment is $2K(d - h) + 3(W - K)$, and $z \equiv 3u \pmod{2K}$. The implementation uses this smaller grid at perfect sizes, with the same asymptotic bound at $s = 0$.

Certify against all other trees. Let C_k be the computed candidate, $a = \min p_i$, $b = \min q_i$, and H' the order-free bound on residual weights. Every excluded tree has $I \geq I_* + k + 1$, so it costs at least

$$B_k = (a + b)(I_* + k + 1) + (a + 2b)n + H'.$$

Thus $C_k \leq B_k$ proves global optimality. Sorting and the two-queue bound cost $O(n \log n)$ and are included in the counters. The constructor uses $k = 2$ and attempts the shortcut only when $s \leq \text{bit_length}(n)$. Its attempted work is $O(n \log n)$; a failed certificate or skipped size falls back to Knuth. The total worst case remains quadratic. Arbitrary growth of k is exponential, and wide shape spaces remain quadratic even for fixed k.

Validation, failed precursor, and prior work. The implementation passed 1,200 restricted-optimum comparisons against all shapes through eight keys with budgets $0, \ldots, 4$, and 3,000 full mixed-weight checks against cubic DP. It separately checks direct tree costs, both residual-bound implementations, and exact fallback. An additional 104 family checks include zeros, geometric weights, spikes, and key-only/gap-only controls. The seeded size sweep is retained. A precursor substituting ordered depth bounds into the old certificate passed 480 exhaustive checks but rescued none of 24 near-uniform fixtures at its 32-iteration budget. This negative experiment did not prove optimal multiplier search useless; its cost and the failed perfect-tree hypothesis motivated expanding the candidate class.

On the 1,023-key periodic fixture with weights 16 and 17, the excess-two DP finds cost 312,154 and excludes other shapes with bound 312,185. It uses 27,570 states, 137,835 transition checks, 6,052 sorting comparisons, and 1,022 merge comparisons, versus Knuth's 523,776 states and 1,046,532 root candidates. At $n = 1{,}024$, a seeded amplitude-four perturbation uses 48,003 states and 240,000 transition checks, plus 13,784 sorting and 1,264 merge comparisons. It certifies cost 19,477,996.

Six matched trials per fixture use all orderings of Knuth, the preceding baseline-price hybrid, and the excess-two constructor. Median excess-two runtime divided by Knuth runtime was 0.23600 and 0.44831 on amplitude-four perturbations at $n = 1{,}023$ and 1,024, and 0.20790 and 0.17101 on the periodic fixture at $n = 1{,}023$ and 2,047. Each certified in all six trials. Ordinary random input at $n = 1{,}023$ failed and cost 1.34949 times Knuth; amplitude-sixteen input at $n = 2{,}047$ failed and cost 1.13631 times Knuth. The skipped-width control at $n = 768$ ran the same Knuth routine and measured 0.93148, illustrating noise rather than a computational gain. These are exploratory M4/Python timings including all preprocessing and reconstruction. Periodic fixtures are repeated identically; random and perturbation inputs vary between trials. All 42 observations are retained.

The dyadic/prefix-code viewpoint and sequential frontier DP are established ideas. Becker [6] gives a sequential mixed-weight DP for a near-minimal *height* restriction in quadratic time. The band above uses excess *total unweighted path length*, a different restriction; it does not improve Becker's stated problem. Priority of this particular potential, band bound, and exclusion combination remains unestablished.

8 A split-height construction covering every input size

The preceding shape band can be wide even at zero excess. We therefore implement the split-height/Monge mechanism already sketched by Karpinski, Larmore, and Rytter in Section 5 of their longer preprint [4]. That section explicitly supplies the fixed-height Monge property and the two-part construction. It is different from Section 5 of the six-page conference version [3]. No priority claim is made for this mechanism. The present experiment makes the tiling, matrix padding, reconstruction, and separate unrestricted certificate explicit.

Two scales of dyadic intervals. Fix an external height limit $H = L + U$ and write $K = 2^L$, $B = 2^U$. Precompute all relative costs $C_h(i, j)$ for $0 \le h \le L$ and $0 \le j - i \le 2^h - 1$, using

$$C_h(i, j) = W(i, j) + \min_r \{C_{h-1}(i, r) + C_{h-1}(r + 1, j)\}.$$

Empty intervals have zero cost; the root range enforces both child height capacities. Monge inequalities at fixed height give monotone minimizing roots as the endpoints move. Choose leftmost ties and bound each root by the adjacent shorter intervals. Search ranges telescope to $O(n)$ for each interval length. Summing over lengths and heights costs $O(n2^L)$.

Divide the unit dyadic interval into B top-grid blocks. Every tree of height at most H consists of lower subtrees contained in individual blocks, together with shallower gaps occupying aligned groups of blocks. Let $D_b(i)$ be the minimum partial cost after consuming i gaps and reaching grid boundary b. Keys through $i - 2$ have already been charged. For $b > 0$, put

$$R_b(i) = D_b(i) + p_{i-1}(U - v_2(b)); \qquad R_0(0) = 0.$$

The boundary key has exactly this comparison length. There are two types of transition:

$$\begin{aligned} D_{b+1}(j) &\leftarrow \min\{D_{b+1}(j), R_b(i) + C_L(i, j - 1) + UW(i, j - 1)\}, \qquad i < j \le i + K, \\ D_{b+w}(i + 1) &\leftarrow \min\{D_{b+w}(i + 1), R_b(i) + q_i(U - \log_2 w)\}. \end{aligned}$$

The second transition requires a power of two $w \ge 2$, $w \mid b$, and $b + w \le B$. Width-one gaps are zero-key lower subtrees in the first transition. Start at $D_0(0) = 0$ and finish at $D_B(n + 1)$. Dyadic alignment proves that these paths represent every height-feasible tree and only such trees. Backpointers recover every lower tree and every boundary key.

Linear matrix searches with feasible padding. At fixed b, the first transition is a banded Monge matrix in i, j; row additions and the separable interval mass preserve the inequality. Reachable counts i are contiguous: the coarsest tiling of the prefix has popcount(b) leaves, and successive refinement gives every count through bK, truncated to n before another transition. Search output columns from $i_{\min} + 1$ through $\min(n + 1, i_{\max} + K)$, each of which has a feasible input row.

SMAWK requires care outside the band. Let $S = (H + 1)(\sum p + \sum q) + 1$, larger than every feasible prefix cost. In the transposed matrix with row j and candidate column i, use

$$A(j, i) = \begin{cases} S + j - K - i, & i < j - K, \\ R_b(i) + C_L(i, j - 1) + UW(i, j - 1), & j - K \le i < j, \\ S + i - j + 1, & i \ge j. \end{cases}$$

This padding preserves total monotonicity. For $i_1 < i_2$, once i_2 is preferred at a row, it remains preferred as j increases: within the feasible band this is Monge; outside it, distances decrease toward the band and increase away from it, and finite entries beat all padding. When the candidates straddle opposite invalid sides, their cost difference is $i_1 + i_2 + K + 1 - 2j$, also decreasing. Ties preserve the same non-strict implication. Thus SMAWK finds leftmost minima in $O(n)$ queries per grid boundary. Padding by tied infinities does not supply this argument.

Across all boundaries there are $B - 1$ aligned choices of coarse width, so the second transitions and scans also cost $O(nB)$. Choosing $L = \min(H, \lfloor H/2 \rfloor + 1)$ gives

$$O(n(2^L + 2^U)) = O(n2^{H/2})$$

work and storage including backpointers. Consequently a fixed extra height $H = \lceil \log_2(n + 1) \rceil + c$ gives $O(n^{3/2})$ attempted work at *every* size. The prototype uses sparse dictionaries; the bounded integer state coordinates also admit direct array indexing.

Lemma 6 (Minimum path length with a tall path)**.** *For $1 \le R \le n$, the minimum internal path length $J(n, R)$ among n-key trees of external height at least R is obtained by forcing a path of R internal nodes and filling the remaining available positions in increasing order of depth. The excluded class is empty if $R > n$.*

Proof. Every tree in the class contains such a path, contributing $R(R-1)/2$. The remaining nodes form an ancestor-closed subset of its free forest: one side root at each depth $1, \ldots, R-1$, and two roots at depth R. The $n - R$ shallowest positions in this forest are ancestor-closed and minimize the depth sum among all subsets of that size. Their addition constructs a feasible tree attaining the bound. Directions of the forced path do not change the forest's depth counts. □

A level-count implementation computes J in $O(R + \log(n+1))$ work.

Theorem 7 (Certificate against taller trees)**.** *Let C_H be the exact height-restricted optimum above, and let Q be the order-free mixed-node Huffman lower bound for residual weights $p - a, q - b$, where $a = \min p$, $b = \min q$. If $H < n$ and*

$$C_H \le (a+b)J(n, H+1) + (a+2b)n + Q,$$

the constructed tree is globally optimal. If $H \ge n$, there is no height restriction and certification is automatic.

Proof. Every excluded tree has internal path length at least $J(n, H+1)$. Its baseline contribution is $(a+b)I + (a+2b)n$ and its residual cost is at least Q. Within the height class, C_H is optimal. □

Sorting and the residual oracle add $O(n \log n)$ work. Failed tests invoke Knuth, so the full constructor remains quadratic in the worst case. The certificate never assumes success from near-uniform weights.

Exact verification and operation evidence. The harness passed 450 exhaustive weighted height-class checks through eight keys, 44 exhaustive tall-path minima, 2,000 cubic-checked mixed inputs with complete fallback (917 successful certificates), and 104 additional adversarial-family inputs. An independent all-root height recurrence checked 520 candidates and 261,482 stored short-table entries; the same 520 cases also compared SMAWK with a retained divide-and-conquer matrix search. The larger sweep has 36 fixtures at three height budgets each. Every near-uniform fixture certified, including previously skipped sizes 384, 768, 1,000, and 1,536. All random, geometric, and gap-spike fixtures failed the certificate, sometimes despite an optimal restricted candidate.

At $n = 1{,}000$, the minimum-height candidate and full optimum are both 18,973,413, below the exclusion bound 18,974,193. The optimizer uses 117,453 short states, 220,483 short root candidates, 84,937 genuine matrix queries, 10,268 padding queries, 16,016 prefix scans, and 5,630 coarse transitions. The bound adds 10,648 sorting and 1,335 merge comparisons, plus 1,000 contractions; all keys are reconstructed. Knuth uses 500,500 states and 1,002,162 root candidates. These distinct operation types are reported separately, without treating them as equal-cost instructions.

Matched timing and failure controls. Six trials per fixture use every ordering of Knuth, the minimum-height constructor ($c = 0$), and one extra level ($c = 1$). Preprocessing and reconstruction are included; every result is checked against Knuth. Inputs vary between trials except the repeated periodic fixture. On the sponsor's M4 with Python 3.9.6, median per-trial runtime ratios were:

Fixture	$c = 0$ ratio	certified	$c = 1$ ratio	certified
384, near-uniform	1.22484	6/6	2.29965	6/6
768, near-uniform	0.94444	6/6	1.40863	6/6
1,000, near-uniform	0.71530	6/6	1.03015	6/6
1,536, near-uniform	0.65826	6/6	1.03916	6/6
3,072, near-uniform	1.46288	0/6	0.70837	6/6
1,536, amplitude four	1.67333	0/6	1.18924	6/6
1,536, ordinary random	1.55378	0/6	1.93841	0/6
1,023, periodic $A = 16$	1.69541	0/6	1.06171	6/6

All 48 observations and operation counters are retained. These are mixed practical results: small successful cases can be slower, and random cases pay substantial overhead before quadratic fallback. At 3,072 near-uniform keys, the minimum-height candidates are optimal but fail certification; one extra level strengthens the exclusion bound sufficiently. For the periodic fixture, minimum height actually excludes the true optimum. The earlier bounded-excess solver is more efficient on that narrow class. No broad practical superiority or improvement over the known width-limited Larmore/KLR constructor is established by this experiment.

9 A variable interval frontier for exact full construction

The height test can fail even when its candidate is optimal. We now use a class guaranteed in advance to contain an unrestricted optimum. The short-interval/top-skeleton architecture is established work of Larmore and Karpinski–Larmore–Rytter [2, 3]; the experiment compares a uniform width envelope with a variable interval contour. Both use the same safe mass guarantee and matrix infrastructure. This is not a priority claim for the architecture or its restricted subquadratic regime.

A failed transfer of the sharper mass constant. The initial implementation used $2W/F_{d+3}$ as the maximum internal subtree mass at edge depth d, following a literal transfer of the Fibonacci formula in [5]. Its first full-input check failed: the resulting class could be empty. The positive two-key example

$$p = (1, 1), \qquad q = (1, 100, 1)$$

explains the failure. Both trees cost 206; the total mass is 104 and the internal child has mass 102. Thus $102 > 2 \cdot 104/3$ at depth one. Charging an additional comparison at every gap is a tree-independent constant and does not remove this violation. We preserve the counterexample and use the following independently justified, weaker constant in our model. This does not refute the KLR architecture or its asymptotic guarantees.

Lemma 8 (A sufficient mixed-weight subtree bound)**.** *If S is an internal subtree at edge depth d in an optimal full OBST of total outcome mass W, then*

$$W(S) \le W/F_{d+1}, \qquad F_1 = F_2 = 1.$$

Proof. Let X be an internal grandchild of v, with parent y, and let O be the mass inside v's subtree but outside y's, including key v. For a straight two-edge path, rotating y upward changes cost by $O - p_y - W(X)$. For a bent path, promoting X with a double rotation changes cost by $O - p_X - W(X)$. Optimality therefore implies $O \ge W(X)$ in either case. These formulas include all external gap weights.

Going upward from S, write $T_0 = S, T_1, \ldots, T_d = T$. Nonnegativity gives $W(T_1) \geq W(T_0)$, and the grandchild inequality gives $W(T_i) \geq W(T_{i-1}) + W(T_{i-2})$ for $i \geq 2$. Induction gives $W(T_d) \geq F_{d+1}W(S)$. Weak inequalities include zero weights and ties. □

This lemma concerns internal subtrees. A heavy interior external gap can have nearly all the mass at depth two; zero-key subtrees remain permitted. The Fibonacci approach is prior work [5]; sharpness of this particular sufficient bound is not claimed.

Store a contour instead of a global width. For fixed d, let $E_d(i)$ be the largest $j > i$ satisfying $F_{d+1}W(i, j) \leq W$, or i if there is none. Empty intervals always have stored cost zero. Nonnegative weights imply that $E_d(i)$ is nondecreasing and the domain is closed under taking subintervals. A two-pointer scan computes it in $O(n)$ work. Write

$$S_d = \sum_{i=0}^{n} (E_d(i) - i).$$

By Lemma 8, every internal subtree at depth d of an optimal tree belongs to the domain. For comparison, the uniform version sets $\ell_d = \max_i(E_d(i) - i)$ and stores

$$U_d = \ell_d(n+1) - \ell_d(\ell_d + 1)/2 \ \geq S_d$$

intervals. Both classes contain an unrestricted optimum.

Lemma 9 (Root-search work on a hereditary contour)**.** *Computing all unrestricted interval optima in this domain takes $O(n + S_d)$ work, with at most $3S_d$ tested roots.*

Proof. Bucket only existing cells by length in $O(n + S_d)$ work. Knuth's recurrence applies because every child and adjacent shorter interval is present. Let $r(i, j)$ be the leftmost optimal root and put $t(i, j) = r(i, j) - i \geq 0$. For length at least two, the root count is $2 + t(i+1, j) - t(i, j-1)$. Sum over the domain. A positive occurrence $t(a, b)$ cancels its negative occurrence whenever $(a, b+1)$ is present. An unmatched positive occurrence requires $b = E_d(a)$ and $E_d(a-1) = E_d(a)$. There is at most one per row, of size at most $E_d(a) - a$. Discard unmatched negative occurrences. Adding the length-one cells proves the $3S_d$ bound. Scanning nonexistent diagonal cells would lose this saving; the buckets are essential. □

Compose the exact interval costs. Use 2^d top-grid blocks and the preceding dyadic transitions, replacing height-limited lower costs by these unrestricted interval costs. A block starting after i gaps can finish after j exactly when $i < j \leq E_d(i) + 1$. It contributes $C(i, j-1) + dW(i, j-1)$ and the starting boundary key cost $p_{i-1}(d - v_2(b))$ at $b > 0$. Coarse gaps retain their aligned power-of-two skip transitions. Every optimal tree is represented, while every reconstructed candidate is a valid full tree. Thus the resulting optimum is unrestricted.

The variable band needs another implementation detail. Reachable input rows, and their union of feasible output columns, may have holes. Scan once per top boundary to retain exactly output columns covered by some reachable row. Put $L(j) = \min\{i : E_d(i) + 1 \geq j\}$, a nondecreasing sequence with $L(j) \leq j - 1$. In the transposed matrix, use finite true entries when $L(j) \leq i < j$, and padding

$$Z + L(j) - i \quad (i < L(j)), \qquad Z + i - j + 1 \quad (i \geq j), \qquad Z = (n + d + 1)W + 1.$$

All finite prefix costs are smaller than Z. Within the band, exact interval costs are Monge. Across padding, opposite-side differences $i_1 + i_2 + 1 - j - L(j)$ are nonincreasing; same-side preferences

also move right. Thus the matrix is totally monotone, even after removing arbitrary rows and columns, and every searched column has a feasible minimum. SMAWK, scans, and coarse transitions together cost $O(n2^d)$. Backpointers recover every key and gap contribution.

Theorem 10 (Exact mass-frontier constructor)**.** *The adaptive implementation constructs a full optimal BST in*

$$O\Big(n\log(n+1)+\min_{d\geq 0}\{S_d+n2^d\}\Big)$$

arithmetic work. The uniform-envelope implementation replaces S_d by U_d. At the selected depth, storage is $O(n+S_d+n2^d)$ or $O(n+U_d+n2^d)$ respectively. Both remain quadratic in the worst case.

Proof. The interval and composition arguments establish exactness at every depth. Start with depth zero and the full Knuth table. Evaluate successive depths while $n2^d$ is smaller than the best work proxy so far; each contour costs $O(n)$. No later depth can improve the proxy because its top term alone is too large. At most $O(\log(n+1))$ depths are evaluated. Depth zero uses Knuth directly, omitting a redundant top pass and its lower-order n proxy term. The preceding work and storage bounds now apply. □

This constructor has no posterior certificate failure followed by a second solve. It may select the full table initially. The Python prototype uses dictionaries; bounded integer top coordinates also allow direct arrays. When each key has weight at least $a > 0$ and $W = O(na)$, we have $S_d = O(n^2/\phi^d)$, where $\phi = (1+\sqrt{5})/2$. Balancing against $n2^d$ recovers the known exponent $1+\log_{2\phi} 2 \simeq 1.59023$.

A scoped asymptotic separation. Let n range over fourth powers and $L = n^{3/4}$. Give every key and gap weight $A = n^2$, except a central block of L keys and its $L+1$ gaps, all of weight one. Every weight is positive and $W = \Theta(nA)$. An admitted interval contains at most $m = O(n/F_{d+1})$ heavy keys. Intervals wholly inside the light block number $O(L^2)$. Other starts contribute $O(nm)$ ordinary endpoints; only $O(m)$ outside starts can reach across a block boundary and gain $O(L)$ endpoints, and starts inside add at most $O(Lm)$ heavy endpoints. Consequently

$$S_d = O(n^2/F_{d+1} + L^2).$$

With $d = \lfloor \log_{2\phi} n \rfloor$, the adaptive work is $O(n^{1+\log_{2\phi} 2} + n^{3/2}) = O(n^{1.59024})$.

Every uniform-envelope plan, however, has proxy $\Omega(nL)$. If $2^d \geq L$, the top term suffices. Otherwise $F_{d+1} = O(L^{\log_2 \phi})$, so W/F_{d+1} exceeds the entire light block's $O(L)$ mass. Hence $\ell_d \geq L$ and the uniform table has at least $nL/2$ cells. This gives $\Omega(n^{1.75})$ at every depth. The separation is within the specified architecture, not against every prior OBST algorithm or combinations of our shortcuts. In particular, this light block also offers an opportunity for certified contraction.

Both modes were checked against Knuth on this positive family at $n = 16, 81, 256, 625$, with cubic checks also at the first two sizes. Larger instances were used only for plan counting. At $n = 65{,}536$, the chosen adaptive and uniform proxies were 119,041,540 and 361,608,522, both at depth nine. These are $S_d + n2^d$ and $U_d + n2^d$, not measured full-solve operation totals or timings. All parameter counts are retained.

Independent verification. The harness passed 960 weighted exhaustive-shape comparisons through eight keys on arbitrary nondecreasing contours, including infeasible classes. It checked 914 interval costs independently and 90,193 top minima by brute search. Another 2,400 full solves,

using both modes on 1,200 inputs through 40 keys, matched cubic DP and reconstructed costs. All 24,600 internal subtrees in the cubic-optimal trees satisfied Lemma 8. Families include zeros, ties, geometric variants, spikes, and key-only and gap-only controls. A further 56 fixtures through 2,048 keys matched Knuth in both modes, including ordinary random inputs that failed the preceding height certificates.

At 2,048 random keys, the adaptive version chose depth six and returned cost 21,053,524. It used 309,261 interval cells and 616,642 root candidates, versus Knuth's 2,098,176 cells and 4,215,527 candidates. Additional work included 24,035 contour mass checks across seven parameter depths, 1,245,431 genuine matrix queries, 149,319 padding queries, 131,136 scans of each row/column type, and 107,859 coarse transitions. The uniform version stored 337,550 cells. On log-uniform weights the corresponding cell counts were 308,487 versus 431,951; on a single heavy key they were 1,048,576 versus 1,573,376. Geometric inputs selected the full table. All operation categories and parameter levels are retained separately.

Matched timings. All six orders of Knuth, uniform width, and adaptive contour are used per fixture, for 48 trials. Preprocessing and reconstruction are timed; every result is directly evaluated and checked against Knuth. Inputs vary except for deterministic geometric and spike fixtures. The timing light window uses weights near 1,000 outside a block of $\lfloor n^{3/4} \rfloor$ keys and weight one inside, with varying position; it is separate from the $A = n^2$ asymptotic family. M4/Python 3.9.6 median paired ratios were:

Fixture	uniform/Knuth	adaptive/Knuth	adaptive/uniform
512, random	1.59908	1.61204	0.96602
1,024, random	1.56592	1.55650	1.00412
2,048, random	0.90002	0.91021	1.00096
2,048, light window	1.13693	0.77546	0.65523
2,048, near-uniform	0.87926	0.91295	1.02587
1,024, log-uniform	1.67408	1.47983	0.88390
512, geometric	0.98060	1.04487	1.01772
2,048, heavy key	1.50542	1.04249	0.70113

The smaller random cases remain slower than Knuth. The contour's saving over uniform width is clearest on localized structure; ordinary random inputs show little timing difference. Both geometric variants execute the same Knuth kernel after selection, so the sub-unit ratio there is noise. These instrumented Python measurements do not establish production-language or broad practical superiority. Every raw observation is retained.

10 Dominant-gap separators and local mass frontiers

The global mass bound can admit many unnecessary intervals when a single heavy outcome dominates otherwise ordinary regions. We first split at certified outcomes, then apply the frontier constructor with each terminal component's own total mass. The following exchanges retain all key and gap weights. Their priority is not established.

Lemma 11 (Restriction to an ordered interval)**.** *A BST restricted to a contiguous key interval and its endpoint gaps has no greater depth for any retained outcome. If the old root is outside the retained key interval, every retained outcome loses at least one comparison.*

Proof. Delete each comparison at an outside key and follow the branch containing all retained outcomes. Comparisons at retained keys remain. This gives the required BST, with each retained

search path a subsequence of its old path. The argument also applies to an empty key interval containing one gap. □

Fix an interior gap g in a current component. Its adjacent keys are $a = g - 1$ and $b = g$. Let L contain the keys strictly left of a, with its endpoint gaps, and let R contain those strictly right of b with its endpoint gaps. Their masses are W_L, W_R, and the component mass is

$$W = W_L + p_a + q_g + p_b + W_R.$$

Theorem 12 (Interior-gap separator)**.** *If*

$$q_g + \min(p_a, p_b) \;\geq\; W_L + W_R, \tag{3}$$

there exists an optimal tree with gap g at depth two. Its top two keys are a, b, in either orientation, and its remaining subtrees are L, R. Writing C_L, C_R for their unrestricted optimal relative costs gives

$$\mathrm{OPT} = C_L + C_R + W + q_g + \min(p_a + W_L,\; p_b + W_R). \tag{4}$$

At equality, other optimal trees need not have this shape.

Proof. Take any tree in which g has depth at least three. Restrict it to L, R using Lemma 11. If its root lies in L, make a the new root, attach restricted L on its left, and make b its right child, with g on b's left and restricted R on its right. Every L outcome deepens by at most one. Every R outcome lost the old root comparison, then gains two, so also deepens by at most one. Key a saves at least one comparison, key b does not deepen, and gap g saves at least one. The cost change is at most $W_L + W_R - p_a - q_g$. For an old root in R, the mirror exchange costs at most $W_L + W_R - p_b - q_g$.

If the old root is a, keep it and promote b to its right child. The L depths do not increase and R depths increase by at most one. As a, b are adjacent and g was deeper than two, b had depth at least two and saves at least one comparison. The change is at most $W_R - p_b - q_g$. The case with old root b is symmetric. Condition (3) makes each bound nonpositive, so the exchange can be applied to an optimum. An interior gap cannot have depth below two; at depth two its bounding ancestor keys must be exactly a, b.

Both orientations leave the same two independent optimization problems. With root a, the cost is $C_L + C_R + W + q_g + p_b + W_R$; with root b it is $C_L + C_R + W + q_g + p_a + W_L$. Minimize these two values after solving each outside interval once. This proves (4) without duplicated recursive branches. □

Replacing the smaller neighboring weight in (3) by the larger one is false. For $p = (100, 1, 50)$ and $q = (1, 1, 1, 1)$, the weakened test accepts $g = 1$ since $101 \geq 53$. Yet the full optimum is 212 and forcing this gap to depth two costs 261. The valid test rejects it since $2 < 53$. This strictly positive counterexample is retained.

Endpoint and key certificates. For a component's first key and left endpoint gap, $2(p_i + q_i) \geq W$ suffices to put that key at the root in some optimum; the right endpoint is symmetric. If its original depth is $h \geq 1$, its endpoint gap has depth $h+1$. Promotion saves $h(p_i + q_i)$ while restricting and reattaching the other interval increases cost by at most $W - p_i - q_i$. Likewise, $2p_r \geq W$ certifies an arbitrary key r as root: promote it and attach the two restrictions, increasing every other depth by at most one while saving at least one comparison on r. These are sufficient conditions, not a general largest-weight root rule.

Avoiding quadratic separator scans. Components remain contiguous intervals of the original input, with no weight aggregation. Store the static scores

$$G_j = 2q_j + p_{j-1} + p_j + \min(p_{j-1}, p_j), \qquad K_j = 2p_j.$$

An interior gap in component $[i, k)$ satisfies (3) exactly when $G_j \geq W(i, k)$, for $i < j < k$. Two range-maximum segment trees find qualifying gaps and keys in $O(\log(n+1))$ work per query, after $O(n)$ construction. Prefix sums give component masses in constant work. Each separator removes one or two keys, and terminal intervals are disjoint. Only terminal slices are copied, so a long chain does not cause quadratic copying or rescanning. The implementation tests endpoints first and builds the indexes only when an endpoint test fails on both sides. All-zero components are returned as balanced trees directly.

Theorem 13 (Local-frontier composition)**.** *For terminal components t with m_t keys and local admissible-interval counts $S_{t,d}$, the constructor is exact and uses*

$$O\left(n \log(n+1) + \sum_t \min_d \{S_{t,d} + m_t 2^d\}\right)$$

arithmetic work. If $m_t \leq b$ for every terminal, the bound is $O(n \log(n+1) + nb)$. Sequential reuse of terminal scratch storage gives $O(n + \max_t\{S_{t,d_t} + m_t 2^{d_t}\})$ space at selected depths d_t. The unrestricted worst case remains quadratic.

Proof. The separator identities reduce the optimum to disjoint exact subproblems. There are $O(n)$ separator calls and $\sum_t m_t \leq n$. The local frontier bounds apply separately, including parameter selection since $\sum_t m_t \log(m_t + 1) \leq n \log(n+1)$. Each terminal may select depth zero and use its full Knuth table, so $\sum_t m_t^2 \leq nb$ when all sizes are at most b. Include the reconstructed output and separator stack in $O(n)$ space; completed terminal scratch arrays need not remain allocated. □

Two exact structured regimes. For $p_i = q_i = 2^i$ and $q_n = 2^n$, the total mass is $3 \cdot 2^n - 2$ and the rightmost key/gap pair has mass $3 \cdot 2^{n-1}$. The right endpoint certificate passes, and the remaining prefix has the same form. The optimal chain is constructed with n endpoint decisions, no range index, and no terminal DP: $O(n)$ arithmetic work. At the final single key the implementation selects the equivalent left endpoint rule. Weights have $O(n)$ bits, so this is not a linear bit-complexity assertion.

For a different positive mixed family, recursively plant a heavy central gap separating two outside intervals, stopping below 32 keys. Set each planted weight to four times its component mass after its children have been planted. Each planted gap satisfies the theorem, and all terminal sizes are below 32. This gives $O(n \log n)$ arithmetic work on the family, including varying positive background weights on both keys and gaps. Neither regime is an unrestricted subquadratic result.

Verification and complete work accounting. The harness exhaustively checks 22,140 weight assignments in $\{0, 1, 2\}$ through four keys against every tree shape. It checks 3,211 qualifying gap certificates and their cost identities, 2,036 endpoint certificates, and 380 key certificates. A further 3,280 full solves, using both frontier and Knuth terminal strategies on 1,640 inputs through 40 keys, match independent cubic costs and direct tree evaluation. Inputs include empty, zero, tied, geometric, spike, key-only and gap-only controls. Both strategies and the direct global frontier also match Knuth on 40 fixtures through 2,048 keys.

At $n = 2048$, the direct and local frontier interval counts are:

Fixture	direct frontier	local frontier	terminal components
Random	308,115	308,115	1
Near-uniform	309,290	309,290	1
Geometric	2,098,176	0	0
Heavy key	1,048,576	153,660	2
Heavy gap	1,047,552	153,582	2
Random background, heavy gap	1,047,552	153,362	2
Hierarchical gaps	522,242	22,333	64

Cell counts do not include all work. On hierarchical gaps, 63 gap separators require 254 endpoint checks, 6,142 index-build comparisons and 1,476 range query comparisons. Terminal solves use 42,885 root candidates, 39,284 genuine top-matrix queries, 6,261 padding queries, 8,122 contour mass checks, 6,952 scans of each row/column type, and 1,792 coarse transitions. Their 1,922 keys are reconstructed and relabeled. On the heavy-gap fixture, 305,118 terminal root candidates and 1,214,092 genuine top queries accompany the reduced interval table. All counter categories are retained; metadata such as local selected depths and actual heights are summed over terminals, not reported as a global tree height. Geometric input uses 2,048 endpoint separators and 4,095 endpoint checks, with no index or terminal work.

Matched timings and competing shortcuts. Ten balanced execution orders compare five constructors per fixture: Knuth, direct frontier, separator with frontier terminals (SF), separator with Knuth terminals (SK), and the earlier recursive contraction solver (RC). These are the five cyclic rotations and their reversals, not all 120 permutations. The 70 trials include 350 timed solves, preprocessing and reconstruction. Every reconstructed cost is checked and all five answers agree. Inputs vary between trials except deterministic spike and geometric families. M4/Python 3.9.6 median paired runtime ratios to Knuth are:

Fixture	frontier	SF	SK	RC
2,048, random	0.87985	0.87172	1.03209	1.00381
2,048, near-uniform	0.87747	0.90863	1.06833	1.06450
2,048, heavy key	0.99362	0.74038	0.52307	0.01605
2,048, heavy gap	0.99617	0.77197	0.51357	1.03871
2,048, random heavy gap	0.90780	0.75211	0.49766	0.97692
2,048, hierarchical gaps	1.06953	0.04608	0.01843	0.99421
1,024, geometric	1.05318	0.00781	0.00548	0.04206

The new gap certificates improve regimes where the earlier contraction solver does little. They do not replace that solver: RC is much faster on the heavy-key fixture. Local frontier construction also costs more than plain Knuth on these smaller terminal components despite storing fewer cells; SK is faster than SF on every separator-active fixture. The geometric SF/SK paths execute the same endpoint logic with no terminal solve, so their timing difference does not reflect an algorithmic distinction. Random and near-uniform fixtures have no separator and expose screening overhead. All raw observations are retained. These instrumented Python results establish scoped empirical benefits, not broad production-language superiority or an asymptotic speedup inferred from timing.

A positive obstruction to the combined constructor. For $n \geq 5$, set every key and gap weight to one except $q_1 = q_{n-1} = M = n^3$. Then $W = 2M + 2n - 1$. The largest gap score is $2M + 3 < W$, endpoint scores are four, and key scores are two, so no separator fires. Indeed the frontier planner selects depth zero and the full table. To see this, write $\tau(k) = k(k+1)/2$ and

$h = \lfloor (n-1)/2 \rfloor$. Depth one admits all $\tau(n)$ cells. At depth two, an interval containing one heavy gap has mass $M + 2\ell$, so the cutoff $W/2 = M + n - 1/2$ admits it exactly for length $\ell \le h$. There are four disjoint sets of h such intervals, and $\tau(n-4)$ intervals avoiding both heavy gaps. Thus

$$S_2 + 4n = \tau(n-4) + 4h + 4n = \tau(n) + 6 + 4h > \tau(n).$$

For $d \ge 3$ with $2^d \le n$, we have $3 \le F_{d+1} \le 2^d \le n$. The cutoff is below M but above every all-unit interval mass, so $S_d = \tau(n-4)$ and

$$S_d + n2^d \ge \tau(n-4) + 8n = \tau(n) + 4n + 6 > \tau(n).$$

If $2^d > n$, the top term alone exceeds $n^2 \ge \tau(n)$. Every positive depth is therefore worse than the direct Knuth plan. This is a quadratic obstruction to the specified constructor, not a general OBST lower bound. An independent contour check rejected the initial assumption that all one-heavy-gap intervals were admitted at depth two; the correct length cutoff above is retained in the test. The family motivates joint treatment of several comparable heavy outcomes. Eight full instances through 256 keys match Knuth and reconstructed costs, with cubic checks through 32 keys. Four larger instances through 65,536 keys check plans only; at that size the selected table has 2,147,516,416 cells, not a completed full solve.

11 Joint heavy-outcome cores

The two-gap obstruction motivates selecting several heavy outcomes together. The following certificate uses their necessary comparison keys as an ancestor-closed core. It preserves the complete mixed-weight objective and solves every intervening interval exactly. Priority of the sufficient condition has not been established.

For $n \ge 1$, choose a nonempty set $\mathcal{H}$ of positive-weight key or gap outcomes. Let A contain every selected key and the adjacent key(s) of every selected gap. Write $m = |A|$, $\theta = \min_{x \in \mathcal{H}} w_x$, and let W_0 be the total weight outside $\mathcal{H}$. Both adjacent keys of an interior gap are ancestors of that gap in every BST. Thus each A-key lies on the search path to some selected outcome, possibly itself. Call a tree *core-closed* if no key outside A is an ancestor of an A-key. Then A forms a top BST and the intervals between consecutive A-keys, including both end intervals, are its payloads. Selected gaps remain singleton core gaps.

Lemma 14 (Projection preserving selected depths)**.** *Every BST can be reduced to a BST on A without increasing any selected outcome's depth. If the original tree is not core-closed, at least one selected outcome becomes strictly shallower.*

Proof. Consider an outside key r whose subtree contains an A-key. If just one child contains A-keys, replace r by that child. The other branch contains no selected outcome: any nonempty subtree containing a selected gap must contain one of its neighboring keys, and a selected empty child gap would make r itself an A-key. All retained selected outcomes save a comparison.

If both children contain A-keys, let a be the largest such key in the left subtree. Replace r by a, restrict the old left subtree to keys below a and their endpoint gaps, and retain the old right subtree. No selected outcome lies strictly between a and r: a selected key would belong to A, and a selected gap would put an adjacent key in A before or at r. All A-keys survive. Selected outcomes in the left restriction lose zero or more comparisons by Lemma 11; right depths are unchanged. If a is selected, it moves strictly upward. Otherwise it must border a selected gap on its left. Its right gap cannot be selected, since that would put $a+1$ in A, contradicting either the maximality of a or

$r \notin A$. The selected left gap loses the comparison at the old a. Thus some selected depth decreases in either case.

These exchanges may discard light keys, which will be reinserted later. Each exchange reduces the number of outside keys and retains A. Repeat until no outside key has an A-descendant, then discard the remaining anchor-free branches. Selected gaps are preserved throughout. This produces the asserted core tree. Arbitrary deletion of nonretained comparisons would not suffice: a nonretained key can separate retained outcomes on both sides; the explicit promotion above resolves that case. □

A first sufficient condition is $\theta \geq (m+L)W_0$, where $L = \lceil \log_2(n+1) \rceil$. Project an optimum and reinsert balanced payloads: the selected cost falls by at least θ, and the new light cost is at most $(m+L)W_0$. The next theorem substantially sharpens the height term using the earlier Fibonacci lemma.

Theorem 15 (Protected-height core certificate)**.** *Let $W_{\mathcal{H}}$ be the selected total weight and*

$$H = \max\{j \in \{1, \ldots, m\} : F_j\theta \leq W_{\mathcal{H}}\}, \qquad F_1 = F_2 = 1.$$

If

$$\theta \;\geq\; (H+L)W_0, \tag{5}$$

there exists a core-closed optimum. If the payload intervals have masses W_t and unrestricted relative optimum costs C_t, then

$$\mathrm{OPT}(\textit{original}) = \mathrm{OPT}(\textit{original A-key weights},\ (W_t)_t) + \sum_t C_t.$$

Equality in the certificate does not force every optimum to be core-closed.

Proof. In a protected-only core instance, keep selected weights and set every other weight to zero. Every internal subtree contains an A-key and therefore a selected outcome of weight at least θ in that subtree. By Lemma 8, a depth-d internal subtree of a protected-only optimum has mass at most $W_{\mathcal{H}}/F_{d+1}$. Its height, including external leaves, is consequently at most H.

Project a non-core-closed full optimum by Lemma 14, saving at least θ in selected weighted cost. Replace the projected core by a protected-only optimum; its selected weighted cost cannot increase, although individual depths may change. Reinsert balanced payloads. Every unselected key comparison length or gap depth is at most $H+L$, so the new light cost is at most $(H+L)W_0$. The old light cost was nonnegative. Condition (5) makes the exchange nonincreasing, establishing the existence of a core-closed optimum.

At core-gap depth d, an exact payload contributes $dW_t + C_t$. The core and payload relative costs therefore optimize independently, proving the identity. Empty intervals contribute zero relative cost and keep their gap weight. The final full-weight core is optimized without a height restriction: H bounds the protected-only witness, not the output core. □

For k equal selected weights, $H = O(\log(k+1))$, whereas m can be $2k$. The certificate thus handles much less extreme dominance than the size-based version. It does not remove unselected weights from the final objective.

A threshold sweep with no repeated interval scans. The implementation selects all outcomes of weight at least a threshold θ. Key i is in A precisely when $\max(p_i, q_i, q_{i+1}) \geq \theta$. Sort these activation weights and all outcome weights. Sweep thresholds upward, grouping equal values. Before each threshold, activate as unanchored each key whose activation weight is strictly smaller. A disjoint-set structure maintains the contiguous payload runs and their total triangular state count. A singleton adds one; merging runs of sizes a, b adds ab. Union by size bounds each find by $O(\log n)$, even without using an inverse-Ackermann analysis.

At each certified threshold, evaluate

$$\tau(m) + \sum_t \tau(k_t), \qquad \tau(k) = k(k+1)/2.$$

Keep the minimum proxy, accepting only a strict improvement on $\tau(n)$. This optimizes a Knuth-state proxy, not actual frontier runtime. Fibonacci values through index n take $O(n)$ additions. Since $W_{\mathcal{H}}/\theta$ is nonincreasing during the upward sweep, a single decreasing Fibonacci index supplies the height bound in $O(n)$ total index decrements; cap it by m. The entire selection uses $O(n \log(n+1))$ arithmetic work and $O(n)$ arithmetic storage. Failure invokes the preceding separator constructor, adding screening work without a failed quadratic solve.

Kernels, reconstruction, and scoped bounds. Both core and payloads use either exact local frontiers or Knuth. Before each kernel, uniform key weights p and uniform gap weights q are recognized. Such a tree costs $(p+q)I + np + 2nq$, where I is its unweighted internal path length. A recursively balanced tree minimizes I and is built in linear work. This familiar uniform-weight shortcut is not claimed as new.

If $K(s)$ bounds the chosen exact kernel on s keys, successful construction costs

$$O\left(n \log(n+1) + K(m) + \sum_t K(k_t)\right), \qquad m + \sum_t k_t = n.$$

Sequential scratch reuse retains only the payload outputs between solves. The general work and space bounds remain quadratic. A uniform residual background and k selected outcomes give $m \leq 2k$ and work $O(n \log(n+1) + k^2)$. If residual weights lie in $[a, Ca]$ for fixed $a > 0$ and C, each payload's local frontier costs $O(s^\alpha)$, where $\alpha = 1 + \log_{2\phi} 2 < 1.59024$. A certified plan then costs

$$O\left(n \log(n+1) + k^2 + \sum_t k_t^\alpha\right).$$

With payload sizes at most B, the last sum is at most $nB^{\alpha-1}$. These are bounds for the certified plan; minimizing the triangular proxy need not minimize this sharper expression.

For the preceding obstruction, $q_1 = q_{n-1} = n^3$ with unit background, $n \geq 5$, we have $m = 4$, $H = 3$, and $W_0 = 2n - 1$. The certificate passes, so the four-key core and uniform payloads take $O(n \log n)$ total work. The previous separator/frontier constructor used all $n(n+1)/2$ cells. Bounded positive mixed background and sufficiently large equal selected gap weights instead give an $O(n^{1.59024})$ plan. This resolves that scoped obstruction; it is not an unrestricted subquadratic theorem.

Independent checks and operation counts. Every nonempty selected-outcome subset and every tree shape through four keys were examined. In all 1,937 non-core-closed cases a core-closed tree has componentwise no-greater selected depths and at least one strict improvement. On 22,140

inputs with weights in $\{0, 1, 100\}$, all 17,054 qualifying nontrivial certificates match the optimum over every shape and the composition identity. Another 2,640 full solves on 1,320 inputs through 32 keys, with both kernels, match independent cubic DP and reconstructed costs. On 520 small inputs the threshold sweep matches direct enumeration of every threshold and payload partition. Both kernels also match Knuth and the previous separator constructor on 30 larger fixtures through 2,048 keys.

At $n = 2048$, the following cell counts include both core and payload solves, or the complete fallback when certification fails:

Fixture	previous separator	joint core/frontier	core keys
Random	309,201	309,201	—
Near-uniform	309,290	309,290	—
Two heavy gaps, unit background	2,098,176	10	4
Two heavy gaps, random background	2,098,176	306,506	4
45 heavy gaps	304,486	23,965	90
45 moderate gaps	304,486	23,695	90
45 mixed heavy outcomes	305,421	24,016	68

The moderate-gap fixture sets each selected weight to exactly $(H + L)W_0 = 42{,}395{,}178$, with $H = 9$, $L = 12$, and $W_0 = 2{,}018{,}818$. The old size-based condition would require $(90 + 12)W_0$ and fails. Work includes 63,549 sorting comparisons, 2,047 Fibonacci additions, 3,024 height checks, 1,958 activations, 1,912 unions, and 1,511 find steps. Kernels use 45,365 root candidates, 116,338 genuine top queries, 23,904 padding queries, 12,336 contour checks, and 17,488 scans each for rows and columns. All 2,048 keys are reconstructed. Local kernel metadata such as heights and chosen depths are sums across calls, not global tree heights. The uniform two-gap case still pays selection, 2,044 payload-node constructions and relabelings, and ten core DP cells; it does not perform only ten total operations.

Matched construction timings and controls. Ninety matched trials compare five solvers in ten balanced orders per fixture: five cyclic rotations and their reversals, not all 120 permutations. All 450 timed solves include preprocessing and reconstruction; their costs agree and every output tree is directly evaluated. CF and CK denote the new core with frontier and Knuth kernels, respectively; S is the preceding separator solver and RC the earlier recursive contraction solver. M4/Python 3.9.6 median paired runtime ratios to Knuth are:

Fixture	S	CF	CK	RC
2,048, random	0.85643	0.86589	1.00373	1.02785
2,048, near-uniform	0.90215	0.85239	1.02368	1.04692
2,048, two gaps, unit background	0.97393	0.00700	0.00551	0.98506
2,048, two gaps, random background	1.01846	0.70676	0.79129	0.99945
2,048, 45 heavy gaps	0.84243	0.08905	0.03125	0.99792
2,048, 45 moderate gaps	0.81550	0.08573	0.03229	0.98979
2,048, 45 mixed heavy outcomes	0.87777	0.08823	0.03204	1.00118
512, random	1.68034	1.76764	1.03732	1.02439
1,024, geometric	0.00648	0.02255	0.02378	0.03135

The new core resolves cases where both earlier solvers remain near Knuth's cost. Plain Knuth is faster on the many small payloads, while the frontier helps on the large random payload. The uniform two-gap CF/CK paths use the same uniform shortcut and tiny core, so their timing difference is not evidence of an asymptotic distinction. Controls expose limitations: the 512-key random

frontier path is slower than Knuth, and joint threshold screening makes the geometric case slower than the existing endpoint-only solver. Failed-core controls execute the same separator kernel; sub-unit paired ratios there do not demonstrate an algorithmic benefit. These instrumented Python measurements do not establish production-language or broad practical superiority. An earlier size-certificate timing batch was explicitly stopped after the height refinement; its incomplete timings are not used here.

Full construction beyond the quadratic-table scale. On the unit-background two-heavy-gap family, full trees were constructed at $n = 512, 2048, 8192, 32768, 65536$. Each exact cost matched an independent cubic solve of the four-key instance

$$p = (1, 1, 1, 1), \qquad q = (1, n^3, 2n - 7, n^3, 1),$$

plus $2I_*(n-4) + 3(n-4)$ for the uniform payload, and direct evaluation of the full reconstructed tree. The uniform closed form was separately checked against cubic DP through 16 keys. At $n = 65536$, the exact cost is 1,407,374,885,846,941. The full new solve took 0.59933 seconds in one exploratory run, with ten core DP cells and fifteen core root candidates. It also used 196,799 sorting comparisons, 65,535 Fibonacci additions, 65,535 height checks, 65,532 activations, 65,531 unions, 65,529 find steps, and 196,609 uniformity comparisons. All 65,532 payload keys were constructed and relabeled, and the four core keys were reconstructed.

The previous method was only planned at this size: it selects 2,147,516,416 cells, and its full solve was not executed. Hence no measured runtime ratio to that solve is claimed. The construction result, the separate plan count, and the arithmetic complexity theorem are distinct evidence. Unbounded integer weights and Fibonacci values also preclude interpreting the arithmetic bound as an established bit-complexity bound.

12 Exact checks and preliminary measurements

The code uses nonnegative integer weights, an independent cubic recurrence, Knuth's recurrence, and direct evaluation of reconstructed trees. The basic harness additionally enumerates all tree shapes for small sizes. CPU experiments ran on the sponsor's M4 Mac mini. The reported quantities are algorithmic counts, not inferred asymptotic bounds from elapsed time.

- Single-key contraction: 22,143 exhaustive weight assignments in $\{0, 1, 2\}$ through $n = 4$, and 6,000 seeded random instances through $n = 20$. Every cost and reconstructed tree matched the cubic oracle.
- Block theorem: 19,486 qualifying interval contractions across 4,500 seeded instances through $n = 16$ matched independent cubic costs.
- Block algorithm: 3,000 random instances through $n = 20$, including fixed and adaptive thresholds, matched the cubic oracle and tree evaluation. Additional small cases check threshold selection against every threshold.

Table 1 records one seeded instance per family at $n = 512$. The baseline has 131,328 states. The block column includes all states inside selected blocks, not just the core. Results from different solvers use their separately recorded seeds and are not paired timing experiments.

In the heavy-separator instance all gaps and ordinary keys have integer weights in $\{1, 2, 3\}$, and 22 randomly selected keys have weight 10^6. The block algorithm retains exactly 22 keys. Its state

Table 1: Preliminary exact DP-state counts, including block solves.

Family	Single-key pass + DP	Adaptive blocks + DP
Uniform mixed weights	131,328	131,328
Near-uniform mixed weights	131,328	131,328
Random mixed weights	128,778	130,817
Log-uniform weights	116,886	94,964
Keys only (control)	92,235	88,508
Sparse heavy separators	—	10,344

reduction is about 12.7-fold, but this is not a claim of a 12.7-fold runtime speedup. On ordinary random mixed weights the gain is small. The single-key pass contracts every key for the tested sequence $p_i = 4^i, q_i = 4^i$, but not for most of the ratio-two sequence. Geometric integer inputs also illustrate why bit complexity must be distinguished from arithmetic-operation complexity.

13 A useful relaxation that failed as a general solver

Allow arbitrary assignment of key weights to internal positions and gap weights to external positions. In this relaxed problem, repeatedly take the two smallest current external weights a, b and the smallest remaining internal weight p, replace them by $a + b + p$, and add that sum to the cost.

To prove the rule, take a deepest internal node of an optimum. Its children are leaves. Swapping the smallest internal weight into it and the two smallest external weights into its children cannot increase cost. Contract that cherry and induct. Sorting and a heap give $O(n \log n)$ time. This elementary extension of the Huffman exchange argument is not claimed as novel.

The relaxed cost is a valid lower bound for full OBST. Equality with a feasible ordered tree certifies optimality. It agreed with an independent shape-enumeration and rearrangement oracle on 22,143 exhaustive and 2,700 random instances in the basic harness.

However, using the relaxation on demand in an exact sparse DP was poor on random inputs. At $n = 128$ it visited roughly 4,300–4,600 exact intervals instead of Knuth's 8,256, but spent approximately 3.9–4.1 million weight comparisons constructing bounds and inspected over 100,000 candidate roots. Fewer exact states did not mean less computation.

Nor does the relaxation's interval matrix inherit the required quadrangle inequality. For $p = (9, 1, 12)$, $q = (5, 1, 18, 10)$, its costs H satisfy

$$H(1, 2) + H(2, 3) = 60 > 54 = H(1, 3) + H(2, 2).$$

Thus applying Knuth-style matrix optimization directly to this bound would be unjustified.

Two additional exact counterexamples exclude tempting local certificates. With $p = (3, 5, 18)$, $q = (13, 14, 4, 8)$, the tree with root 2 and left subtree $0(\varnothing, 1)$ costs 127, every single-rotation neighbor costs at least 134, yet the balanced tree costs 125. With $p = (18, 8, 18)$, $q = (17, 11, 1, 13)$, the balanced tree costs 164 and every distinct tree obtained by promoting a descendant to the root of an existing subtree costs at least 169; the tree $0(\varnothing, 2(1, \varnothing))$ costs 157. These are barriers to those local certificates, not complexity lower bounds.

14 An ordered depth-chain lower bound

Let l_i be the comparison length of key i and g_i the depth of gap i. For every valid BST, adjacent keys have different lengths and

$$g_0 = l_0, \quad g_n = l_{n-1}, \quad g_i = \max(l_{i-1}, l_i) \quad (0 < i < n).$$

This identity is existing work; see Becker [6], Lemma 1. Let K_d count keys of length d and G_d count gaps at depth d. The root and child-count identities are

$$K_1 = 1, \qquad G_d + K_{d+1} - 2K_d = 0 \quad (1 \le d \le n), \qquad K_{n+1} = 0.$$

Permit any sequence in $\{1, \dots, n\}^n$ with unequal adjacent entries. For arbitrary multipliers $\lambda_0, \dots, \lambda_n$, minimize

$$\sum_i p_i l_i + \sum_i q_i g_i + \lambda_0(K_1 - 1) + \sum_{d=1}^{n} \lambda_d(G_d + K_{d+1} - 2K_d). \tag{6}$$

Call the minimum $L(\lambda)$. Every actual tree is permitted and has zero penalty, so $L(\lambda) \le \mathrm{OPT}$ for every multiplier vector. The expression is a chain with key energy $U_i(b) = p_i b + \lambda_{b-1} - 2\lambda_b$, gap energy $V_i(b) = q_i b + \lambda_b$, and constant $-\lambda_0$. For an interior transition into length b, take the smaller of

$$\min_{a<b} F(a) + V_i(b), \qquad \min_{a>b}\{F(a) + V_i(a)\}.$$

Prefix and suffix minima give $O(H)$ work per key for a domain of H lengths. Using $H = n$ would cost quadratic time even for one bound evaluation.

Theorem 16 (Exact affine-tail compression)**.** *Fix $h \ge 1$, arbitrary $\lambda_0, \dots, \lambda_h$, and a slope $0 \le s \le \min_i p_i$. Extend $\lambda_d = \lambda_h + s(d - h)$ for $d > h$. The minimum in (6) is unchanged if lengths are restricted to $1, \dots, \min(n, h + 3)$.*

Proof. For $b \ge h + 1$, the unary key energy has slope $p_i - s \ge 0$ and the gap energy has slope $q_i + s \ge 0$. Both are nondecreasing in this tail. If some $l_i > h + 3$, replace it by a member of $\{h + 1, h + 2, h + 3\}$ unequal to its current neighbors. At most two values are forbidden. The key energy does not increase. Each adjacent maximum is either unchanged or decreases within the affine tail, so neither gap energy increases. Endpoint gaps obey the same argument. Repeat for all large lengths. The restricted domain is a subset of the original domain, and every original sequence has a no-more-expensive restricted sequence; therefore the minima are equal. □

Taking $h = \lceil \log_2(n+1) \rceil + 2$ gives an $O(n \log n)$ oracle. This is a statement about minimization of the relaxed energy, with no height assumption on the unknown optimal BST. Optimizing only the head multipliers requires folding all tail constraint violations into the derivative for λ_h. The implementation uses exact integer multipliers and costs scaled by 1024. Its subgradient ascent is heuristic; the bound is valid at every iteration regardless of convergence.

A stronger oracle requires exactly one length1 key. For each candidate root, the left and right chains then contain only lengths at least2. One forward and one reverse pass supply their closing costs for every root position, giving all forced-root bounds in $O(nH)$ total time. The projection above preserves the unique root, so tail compression remains valid for each marginal. Retaining the maximum bound seen for each root, then taking the minimum over roots, is also a valid lower bound.

Both oracles passed 50 comparisons against exhaustive length-sequence minimization, 5,000 arbitrary-multiplier bound checks against cubic OBST, and 1,050 direct comparisons of the full and shortened oracles. Forced-root marginals were additionally checked against exact root-constrained costs.

A structural gap. The sequence $l = (3, 1, 2, 3, 2)$ has $g = (3, 3, 2, 3, 3, 2)$, a unique root, and all the level-count equalities. It is not a valid BST depth sequence. For

$$p = (3, 27, 21, 16, 12), \qquad q = (18, 22, 27, 22, 12, 22),$$

it costs 470, while the true optimum is 472. Its penalty is zero for every multiplier vector. Hence neither oracle family can ever exceed 470 on this instance. More optimization iterations cannot remove this obstruction.

Preliminary strength and cost. At $n = 256$, normalized gaps $(\mathrm{OPT} - \mathrm{LB})/W$ for the chain bound versus order-free Huffman were 0.15003 versus 0.22080 on a random mixed instance, and 0.00873 versus 0.49294 on an alternating instance. However, 300 ascent iterations used 1,082,368 key-depth cells including initialization, compared with 32,896 interval states for Knuth. The unique-root variant uses two passes and still leaves 254 of 256 potentially improving roots on the random instance, and 255 on the alternating instance. Only two survive on the gap-spike instance. These results identify a lower-bound component and its limitations; they do not establish a faster general exact solver.

15 Exact depth ordering and an arbitrary-window obstruction

Call a positive integer sequence *ranked* if every nonempty contiguous interval has a unique minimum. Equivalently, equal labels have a smaller label between them. This is vertex ranking on a path with the label order reversed; vertex ranking and bounded-path variants are existing notions [7, 8]. The following results explain the ordering information absent from the preceding relaxation; no novelty claim is made for this characterization.

Theorem 17 (Depth characterization)**.** *A sequence $l \in \{1, \ldots, n\}^n$, $n \geq 1$, is the inorder key-comparison-length sequence of a BST if and only if it is ranked, $K_1 = 1$, and every level-flow equality holds, with gaps derived from l as above.*

Proof. Necessity follows from the unique shallowest key in each inorder interval and the two children of every internal node. For sufficiency, delete all labels at most d. Let U_d count the remaining nonempty runs, and let A_d count the deleted labels. If the deleted labels form R runs and include e endpoints, then the number of gaps of length at most d is $A_d - R + e$, whereas $U_d = R + 1 - e$. Thus

$$G_{\leq d} = A_d + 1 - U_d.$$

Summing the level-flow equalities from 1 through d, and using $K_1 = 1$, also gives

$$G_{\leq d} = A_d + 1 - K_{d+1}.$$

Hence $U_d = K_{d+1}$. Ranking permits at most one label $d + 1$ per remaining run, so equality forces exactly one. Starting with the unique length1 root, every remaining nonempty interval therefore receives its root at the next level. This constructs the tree with exactly the supplied lengths. □

Ranking has a linear recognizer: scan with an increasing stack, pop larger labels, reject an equal top, and otherwise push. A rejected pair has no smaller intervening label; conversely every such pair causes rejection. Each entry is pushed once and popped at most once. This establishes cheap recognition, not cheap minimization over all valid sequences.

Theorem 18 (Bounded-window obstruction)**.** *For every integer $w \geq 2$ and $t \geq 2$, there is a strictly positive full-OBST instance with $n = w + 3$ keys and an inadmissible depth sequence satisfying all level-flow equalities, a unique length1 root, and ranking on every interval of at most w keys. Its cost is at most $(7/8 + 1/(8t))\,\mathrm{OPT}$. Consequently every lower bound obtained by pricing the level-flow equalities over this window-constrained domain has the same upper limitation, regardless of the multipliers.*

Proof. Take

$$l = (3, 1, 2, 3, 4, \ldots, w + 1, 2).$$

The ascending middle portion has $w - 1$ entries. Its two enclosing length2 labels form the only interval with a repeated minimum, of width $w+1$. Every shorter window passes. Direct substitution into the gap formula verifies $K_1 = 1$ and every level-flow equality. The sequence is invalid: the unique root is key1, but its singleton left subtree has length3, and two keys to its right both claim length2.

Give every gap weight1 and every key weight1 except

$$p_1 = 3B, \qquad p_2 = p_{n-1} = B.$$

The unit-weight outcomes contribute $E = w^2 + 4w + 10$ in the inadmissible sequence, whose cost is $7B + E$. Every actual tree costs at least $8B$. If key1 is the root, the two B-weight keys lie to its right and their comparison lengths sum to at least5, giving $3B + 5B$. Otherwise key1 has length at least2 and each B-weight key has length at least1, giving $6B + 2B$.

Set $B = tE$. The inadmissible cost satisfies

$$7B + E = \left(\frac{7}{8} + \frac{1}{8t}\right) 8B \leq \left(\frac{7}{8} + \frac{1}{8t}\right) \mathrm{OPT}\,.$$

Its level-flow penalty is zero for every multiplier vector, so every priced minimum is at most this cost. □

For $t = 16$ the upper ratio is $113/128$, independently of w; as t increases it approaches $7/8$. This rules out universal exactness of the specified window-minimum repair, not other local constraints, adaptive nonlocal cuts, or subquadratic full-OBST algorithms. Equivalently, at every $n \geq 5$ the construction passes all windows of up to $n - 3$ keys.

Burying the obstruction below a forced prefix. Let a current instance have mass W and a feasible tree of cost U. Prepend a key of weight $A = W + U + 2$ and a new leftmost gap of weight1. Using the new key as root costs $A + 1 + W + U = 2A - 1$, while every other root costs at least $2A$ from that key alone. Hence the new root is forced and $\mathrm{OPT}_{\mathrm{new}} = A + 1 + W + \mathrm{OPT}_{\mathrm{old}}$. The transformation $l \mapsto (1, l_0 + 1, \ldots, l_{n-1} + 1)$ preserves zero flow and local window ranking, and changes the inadmissible cost by the same $A + 1 + W$. Repeating this D times places the strict gap beneath the true optimal top-D right-chain prefix. Thus fixing any constant number of top levels cannot make the remaining window/flow relaxation exact. The constant relative gap is not claimed after this embedding, because the forcing weights add a potentially large common cost.

Independent verification. The harness enumerated all 873,612 sequences in $\{1, \ldots, n\}^n$ for $1 \leq n \leq 7$. The characterization accepted exactly the independently generated BST depth sequences: respectively 1, 2, 5, 14, 42, 132, and 429. The stack recognizer agreed with direct interval scans on every sequence. Width3 and exact flow already admit four invalid sequences at $n = 6$ and

34 at $n = 7$. The positive family was checked at $t = 16$ for all 127 widths $2 \le w \le 128$: exact costs, window validity, zero flow, and the tree-cost lower bound all passed. Knuth and direct tree evaluation checked every instance; independent cubic DP also checked $w \le 24$. Another 120 cases combine $2 \le w \le 16$ with forced prefix depths $1 \le D \le 8$. Cubic DP, Knuth, direct costs, and inspection of the optimal prefix verify the exact additive identity and preserved strict gap.

16 Status and reproducibility

The general subquadratic question remains open. This checkpoint provides exact contraction, bounded-excess and split-height constructions, variable interval frontiers, dominant-gap separators, joint heavy-outcome cores, global certificates, and scoped obstructions to depth relaxations. Mathematical novelty, broad practical benefit, and comparisons against other optimized prior implementations remain to be assessed.

Standard-library Python code, seeded exact results, and matched timing observations are under `obst/code` and `obst/results`. The README gives reproduction commands. The active objective remains to avoid most of the quadratic table on a substantially broader class, or establish a rigorously scoped complexity lower bound.

References

[1] D. E. Knuth. Optimum binary search trees. *Acta Informatica* 1, 14–25, 1971. doi:10.1007/BF00264289.

[2] L. L. Larmore. A subquadratic algorithm for constructing approximately optimal binary search trees. *Journal of Algorithms* 8(4), 579–591, 1987. doi:10.1016/0196-6774(87)90052-6.

[3] M. Karpinski, L. L. Larmore, and W. Rytter. Sequential and parallel subquadratic work algorithms for constructing approximately optimal binary search trees. *Proceedings of SODA*, 36–41, 1996. Author-hosted manuscript.

[4] M. Karpinski, L. L. Larmore, and W. Rytter. Same title as [3], longer preprint version, Section 5: Short height limited trees. Archived preprint text.

[5] D. S. Hirschberg, L. L. Larmore, and M. Molodowitch. Subtree weight ratios for optimal binary search trees. Technical Report 86-02, University of California, Irvine, 1986. Author-hosted report.

[6] P. Becker. Optimal binary search trees with near minimal height. 2010. arXiv:1011.1337.

[7] D. C. McDonald. On-line vertex ranking of trees. *SIAM Journal on Discrete Mathematics* 29(1), 145–156, 2015. arXiv:1401.2669.

[8] P. Bose, V. Dujmović, M. Javarsineh, and P. Morin. Asymptotically optimal vertex ranking of planar graphs. 2022 version. arXiv:2007.06455.